04/08/2026

CSA1402 - Compiler Design [CO2 AT2]

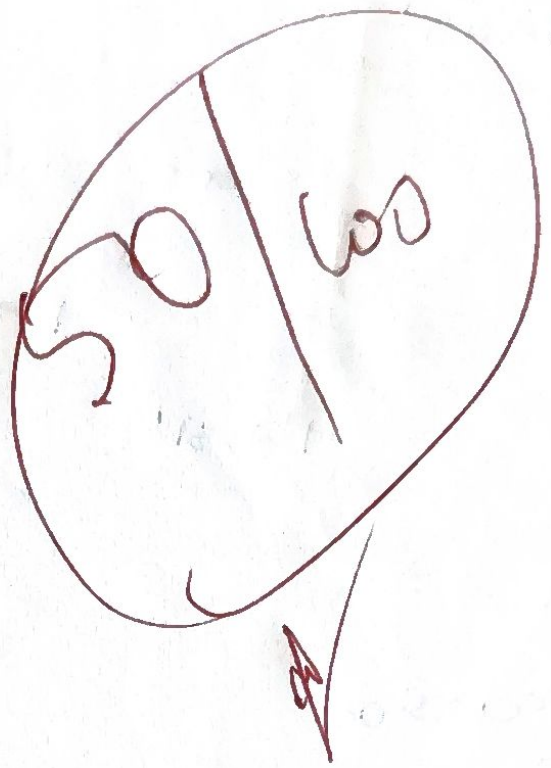
Deepak Rao.L
192421170

Short Answer Test

2) Predictive parser

$S \rightarrow (L) / a$

$L \rightarrow L, S / S$



Step 1: Left Recursion

$S \rightarrow aS'$

$L' \rightarrow ,SL' / \varepsilon$

$L \rightarrow S$

$S' \rightarrow (L) / \varepsilon$

Step 2: find First ()

First (S) = {a}

First (L') = {, , ε}

First (L) = {(,), ε}

First (S') = {(,), ε}

4+1

Step 3: find Follow ()

Follow = $\alpha B \beta$

Follow (S) = {$, , }

Follow (S') = {$, , }

Follow (L) = {$, , ,)}

Follow (L') = {$, , ,)}

Step 4: Predictive parser table

NT / T	a	,	(	)	$
S	S→aS'				
S'		S'→ ε	S'→(L)	S'→(L)	S'→ ε
L			L→S	L→S	
L'		L'→,SL'		L'→ ε	L'→ ε

4) Left Recursion

S → Sa / Sb / c

S → Sa

S → Sb

S → c

S → αB + β

Left Recursion

S → aS' / ε

S → bS' / ε

S → c

Tokens

Degree - F ⇒ Variable (id_1)

= ⇒ Argument Operator

Degree - C ⇒ variable (id_2)

* ⇒ Operator

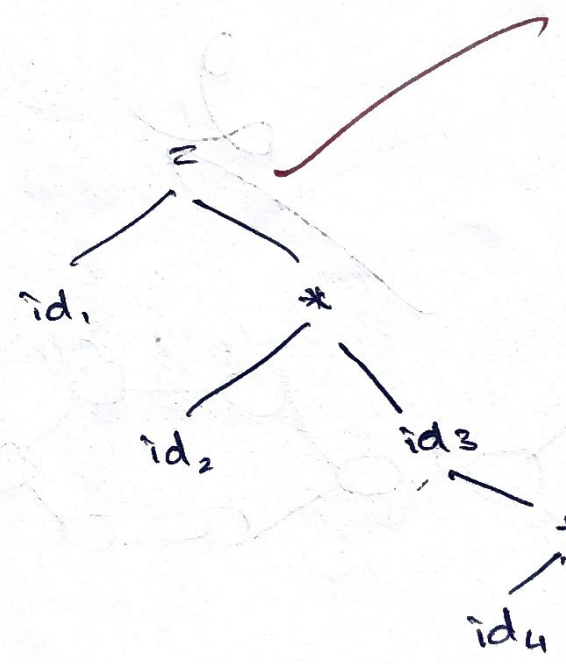
1.8 ⇒ float

+ ⇒ Operator

32 ⇒ Integer

Position = Rate * Initial + final

Syntax phase



2

3) left factoring

A → abcD / abcE / F

A → abcD A → abcE A → F

A → ABCd

A → ABCE

A → F

3) $S \rightarrow SS+ / SS* / a$ string: "aa+a"

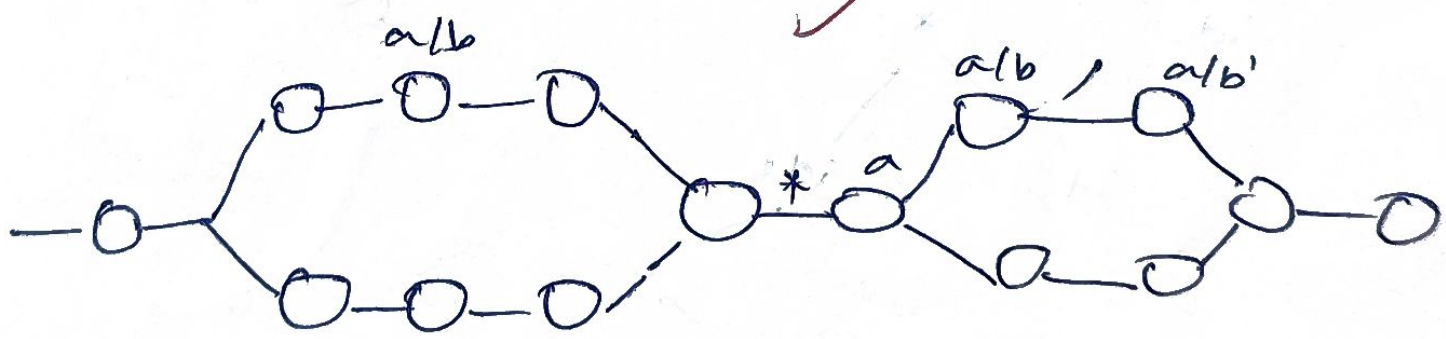
No, is not possible to change or modify grammar to construct predictive parser for the language of

$S \rightarrow SS+ / SS* / a$

1) $(a/b)^* a (a/b) / (a/b)$

Regular Expression

(NFA)



7)

Sum = sum + i;

Lexical Analysis, Syntax Analysis, Semantic Analysis, Intermediate code, code generation, code optimization

Lexical Analysis

Sum → id_1

= → Operator

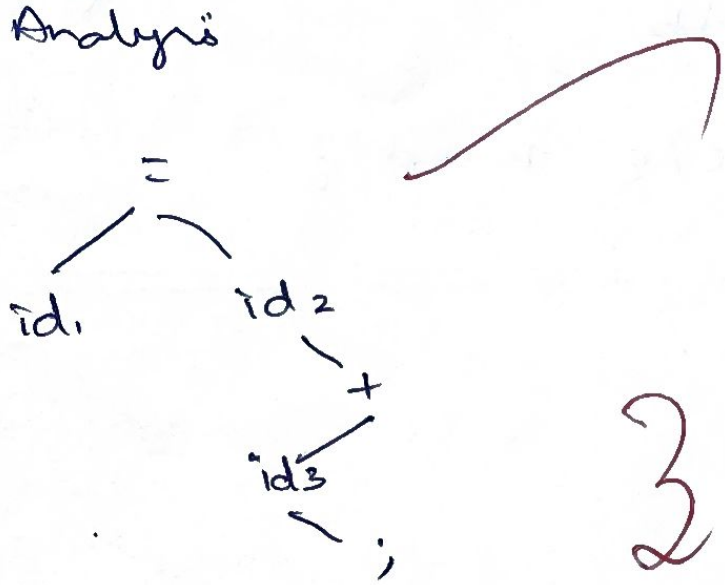
Sum → id_2

+ → Operator

i → id_3

; → Operator

Syntax Analysis



3

Semantic Analysis

Code optimize, code generation

---

8) $S \rightarrow aABb$, $A \rightarrow c/\varepsilon$ $B \rightarrow d/\varepsilon$

First(S) ⇒ {a, b}

First(A) ⇒ {c, ε}

First(B) ⇒ {d, ε}

Follow(S) = {$}

Follow(A) = {$, d}

Follow(B) = {$, d}

Parser table

NT/T	a	b	c	d	$
S	S→aABb	S→aABb			
A			A→c	A→ε	A→ε
B				B→d	B→ε

Parser move ~~dbd~~ acdb

5

S→aABb A→c/ε B→d/ε

9) Lex Program

```
%{
#include <iostream>
if(s=+,+,-,*,/,,,)
count==1
else
 count==0
%
}
```

2

Output

2

Input

+,-

7)

Rule of Eliminate Left Recursion & Left Refactoring

*) Consider a grammar eg: $E \rightarrow E+T$

*) we want to find E' for E using $E \rightarrow \alpha B + \beta$

$E' \rightarrow +TE' / \varepsilon$

*) Add ε atlast for empty set

*) If it is " / " we need to split into two statement

eg: $E \rightarrow E+T / T \Rightarrow E \rightarrow E+T \quad E \rightarrow T$

*) follow these steps to attive a left recursion value.

Left Factoring

*) Consider a grammar eg: $A \rightarrow abcD / abcE / F$

*) First, we need to analyze a grammar context properly

*) If there is any same form or grammar context

$AA' \rightarrow \varepsilon$

*) It is a " / " we need to split three solution grammar

eg: $A \rightarrow abcD$, $A \rightarrow abcE$, $A \rightarrow F$

*) we finally get the left Factoring grammar

11) Shift Reduce Parser

a) First, we have a Context / Argumented free Grammar Context eg: E → E+T/T and string input id * id + id

b) We Add $ on last of the input string like id * id + id $

c) There are four action in shift Reduce Parser

Shift → Compare to a input string and right hand side of the argumented grammar is small we need to Shift action is perform (eg:

$ E	* id + id $	shift

2 Reduce → Compare to a input string and right hand side of the argumented grammer is Big. we need to Reduce action is perform (eg:

$ id	* id + id $	Reduce E → id

Accept → when the both strings are matching most with dollar symbol accept will be perform (eg:

$	$	Accept

Error → If ; it is not matching. Inlast with dollar symbol error action will be perform (eg:

$	id	error

13) $E \to E+T / T \quad T \to T*F / F \quad F \to (E) / id$

Eliminate left recursion ∴

$E \to +TE' / \varepsilon$ $T \to *FT' / \varepsilon$	$E \to T$ $T \to F$	$F \to$

~~First (E)~~

14) Shift Reduce Parser

$S	abcc$	
$AB	abcc$	Reduce [A → AB]
$aB	abcc$	Reduce [A → a]
$B	bcc$	Same
$CB	bcc$	[B → CB]
$CCB	bcc$	[B → CB]

No is NOT coming for these grammar

15) shift reduce parser

$ S	abbc $	
$ aAB	abbc $	Reduce [B → aAB]
$ AB	bbc $	Cancel
$ bAB	bbc $	A → bA
$ AB	bc $	Cancel
$ bB	bc $	A → b
$ B	c $	Cancel
$ C	c $	B → c
$	$	Accept

8

19) S → ABC
A → a / ε
B → r / ε
C → b / ε

3

First (S) = {a, ε}
First (A) = {a, ε}
First (B) = {r, ε}
First (C) = {b, ε}

Follow (S) = {$}
Follow (A) = {$}
Follow (B) = {$, b}
Follow (C) = {$, b}